Zadanie nr 2 - Sieć MADALINE do rozpoznawania znaków

Inteligentna Analiza Danych

Jędrzej Małaczyński 264351 Jędrzej Kostyk 264339

26 października 2026

1 Cel zadania

Celem zadania było zbudowanie prostego systemu rozpoznawania znaków opartego na jednowarstwowej sieci neuronów liniowych MADALINE. System składa się z dwóch programów: `font_generator.py`, który tworzy binarne obrazy pojedynczych liter o zadanym poziomie zaszumienia, oraz `madaline_ocr.py`, który na podstawie obrazów wzorcowych buduje sieć i rozpoznaje obrazy testowe. Sprawdzono poprawność działania systemu (znaki testowe bez szumu) oraz jego skuteczność dla znaków zaszumionych w stopniu 10%, 30%, 50% i 90%, a także zbadano, od czego zależą błędy rozpoznawania.

2 Wstęp teoretyczny

Dla wektorów o długości równej jeden iloczyn skalarny jest równy kosinusowi kąta między nimi [1]:

$$\mathbf{w} \cdot \mathbf{x} = \|\mathbf{w}\| \, \|\mathbf{x}\| \cos \angle(\mathbf{w}, \mathbf{x}) = \cos \angle(\mathbf{w}, \mathbf{x}). \tag{1}$$

Obraz binarny zapisany jako wektor pikseli (czarny = 1, biały = 0) ma wyłącznie nieujemne współrzędne, więc po normalizacji iloczyn (1) leży w przedziale $[0, 1]$. Wartość 1 oznacza obrazy identyczne, a 0 obrazy bez ani jednego wspólnego czarnego piksela. Iloczyn skalarny jest więc miarą podobieństwa kształtów.

Sieć MADALINE (*Multiple ADAptive LINear Elements*) [3] to jedna warstwa k neuronów liniowych, z których każdy otrzymuje ten sam wektor wejściowy. W zastosowaniu do rozpoznawania znaków [1] wektor wag neuronu l ustawia się na znormalizowany wektor pikseli litery wzorcowej l, a wejściem jest znormalizowany wektor pikseli znaku testowego $\hat{\mathbf{x}}$. Sieć nie wymaga więc treningu iteracyjnego, bo wagi są gotowymi wzorcami. Wyjścia neuronów i wynik rozpoznawania to

$$y_l = \mathbf{w}^{(l)} \cdot \hat{\mathbf{x}}, \qquad l^* = \arg\max_l y_l, \qquad cnf = y_{l^*}, \tag{2}$$

gdzie l^* jest indeksem neuronu zwycięskiego, a współczynnik pewności cnf jest kosinusem kąta między wzorcem zwycięzcy a znakiem testowym [2].

Model szumu. Materiały nie podają wzoru na zaszumienie, przyjęto więc, że przy poziomie p (w procentach) każdy piksel z prawdopodobieństwem $p/100$ otrzymuje losową wartość 0 lub 1. Poziom 0% daje obraz czysty, a 100% obraz całkowicie losowy. Pojedynczy piksel zmienia się zatem z prawdopodobieństwem $q = p/200$. Model ten wybrano, bo odtwarza przykładowe wartości pewności podane w materiałach (0,83 dla 10%, 0,61 dla 30%, 0,38

dla 70%). Alternatywa, czyli odwracanie $p\%$ pikseli, przy $p > 50\%$ dawałaby negatyw litery, a nie szum.

Jeśli wzorzec ma n czarnych pikseli na obrazie o $N = w \cdot h$ pikselach, to średnio $n(1-q)$ z nich pozostaje czarnych, a $(N-n)q$ białych staje się czarnymi. Oczekiwane wyjście neuronu właściwej litery wynosi więc w przybliżeniu

$$\bar{y} \approx \frac{n(1-q)}{\sqrt{n}\,\sqrt{n(1-q)+(N-n)q}} = (1-q)\sqrt{\frac{n}{n(1-q)+(N-n)q}}. \qquad (3)$$

Dla obrazu całkowicie losowego ($q = \frac{1}{2}$) wyjście dowolnego neuronu j wynosi średnio $\sqrt{n_j/2N}$. Zależy więc tylko od liczby czarnych pikseli we wzorcu, a nie od kształtu znaku testowego: wygrywa litera z największą liczbą pikseli.

3 Eksperymenty i wyniki

Wspólne założenia wszystkich eksperymentów:

- 26 małych liter alfabetu angielskiego `a`–`z`, czyli sieć ma 26 neuronów,
- krój Times New Roman (`times.ttf`), obrazy 128×128 px, litera w punkcie $(28, 10)$, parametry jak w scenariuszu z materiałów [2]; rozmiar czcionki 102 px (największy, przy którym cała litera mieści się w obrazie),
- wzorce (wagi sieci) to litery bez szumu, katalog `training_patterns`.

3.1 Eksperyment nr 1 – testy poprawności i skuteczności

3.1.1 Założenia

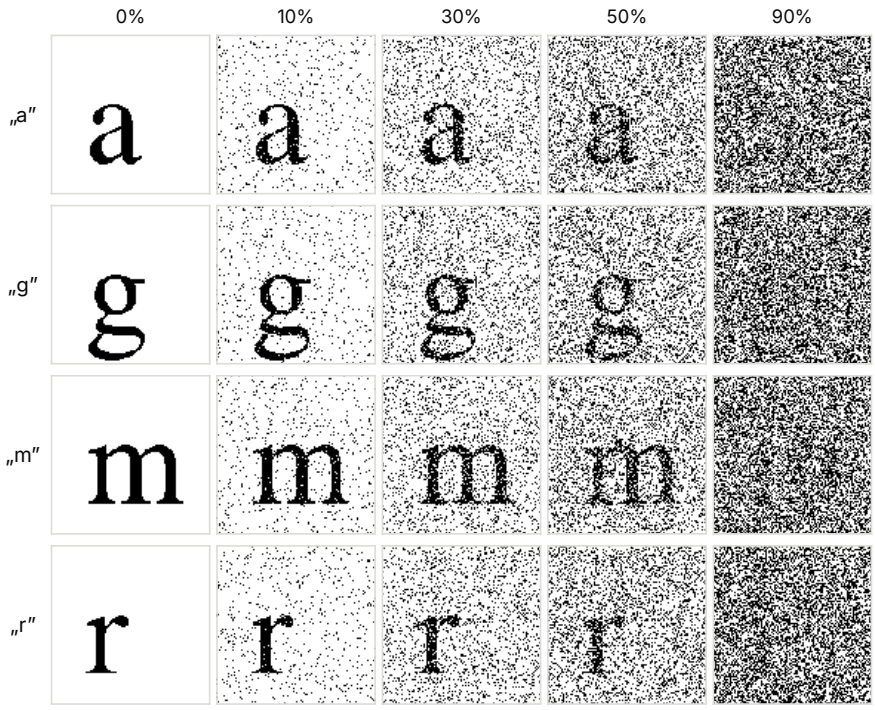
Test wymagany specyfikacją: znaki testowe o zaszumieniu 0% (poprawność) oraz 10%, 30%, 50% i 90% (skuteczność). Dla każdego poziomu wygenerowano jeden komplet 26 liter, a ziarno szumu ustalono jako $1000 \cdot p$+numer litery.

3.1.2 Przebieg

Dane przygotowano wyłącznie poprzez wywołania programów z linii poleceń, tak jak w scenariuszu z materiałów, np.:

```
font_generator.py 128 128 28 10 times.ttf a 0 training_patterns
font_generator.py 128 128 28 10 times.ttf a 30 test_patterns_30 --seed 30000
madaline_ocr.py training_patterns test_patterns_30 > raport_szum_30.txt
```

Wyniki zapisano w plikach `raport_szum_0.txt` … `raport_szum_90.txt` oraz `raport_wzorce_wzorce.txt` (wzorce rozpoznawane same przez siebie). Przykładowe obrazy testowe pokazano na Rys. 1.



Rysunek 1: Przykładowe obrazy testowe dla poziomów zaszumienia 0–90%

3.1.3 Rezultat

Początek raportu dla zaszumienia 30% (format zgodny ze specyfikacją):

```
letter a, noise level 30% -> letter a, noise level 0%, confidence: 0.449
letter b, noise level 30% -> letter b, noise level 0%, confidence: 0.497
letter c, noise level 30% -> letter c, noise level 0%, confidence: 0.400
```

Dla znaków niezaszumionych wszystkie 26 liter rozpoznano poprawnie ze współczynnikiem pewności dokładnie 1,000 (Tab. 1). Ten sam wynik daje uruchomienie `madaline_ocr.py training_patterns training_patterns`. Potwierdza to poprawność implementacji: obraz identyczny ze wzorcem daje kąt zerowy.

Przy zaszumieniu 10%, 30% i 50% również wszystkie litery rozpoznano poprawnie, choć średnia pewność spadła z 0,695 do 0,325. Litery z liniami bardziej cienkimi (np. `i`, `r`, `t`) mają niższą pewność niż litery „ciężkie” (np. `m`, `g`), co zgadza się ze wzorem (3): im mniejsze n, tym większy udział szumu

Tabela 1: Współczynnik pewności dla każdej litery; przy błędzie podano literę rozpoznaną (pogrubioną)

Litera	**0%**	**10%**	**30%**	**50%**	**90%**
a	1,000	0,688	0,449	0,311	**m** (0,210)
b	1,000	0,743	0,497	0,365	**m** (0,226)
c	1,000	0,666	0,400	0,292	**m** (0,214)
d	1,000	0,755	0,497	0,373	0,231
e	1,000	0,674	0,428	0,323	**m** (0,214)
f	1,000	0,662	0,427	0,300	**m** (0,209)
g	1,000	0,759	0,505	0,378	0,223
h	1,000	0,750	0,503	0,386	0,227
i	1,000	0,631	0,373	0,259	**m** (0,204)
j	1,000	0,692	0,423	0,319	**m** (0,204)
k	1,000	0,738	0,486	0,364	**m** (0,215)
l	1,000	0,637	0,394	0,280	**m** (0,216)
m	1,000	0,781	0,545	0,415	0,242
n	1,000	0,714	0,480	0,339	**m** (0,227)
o	1,000	0,687	0,438	0,307	**m** (0,213)
p	1,000	0,737	0,506	0,366	**m** (0,223)
q	1,000	0,743	0,495	0,372	**m** (0,215)
r	1,000	0,613	0,370	0,272	**m** (0,216)
s	1,000	0,642	0,393	0,272	**m** (0,207)
t	1,000	0,635	0,393	0,266	**m** (0,191)
u	1,000	0,709	0,461	0,344	**m** (0,226)
v	1,000	0,628	0,380	0,284	**m** (0,202)
w	1,000	0,736	0,486	0,364	**m** (0,217)
x	1,000	0,681	0,425	0,300	**m** (0,206)
y	1,000	0,687	0,427	0,314	**m** (0,204)
z	1,000	0,671	0,405	0,290	**m** (0,206)
popr.	26/26	26/26	26/26	26/26	4/26
śr.	1,000	0,695	0,446	0,325	0,215

w wektorze wejściowym. Przy 90% poprawnie rozpoznano tylko 4 litery (`d`, `g`, `h`, `m`), a aż 22 z pozostałych rozpoznano jako `m`. Na obrazie widać już praktycznie sam szum (Rys. 1).

3.2 Eksperyment nr 2 – skuteczność w funkcji poziomu szumu

3.2.1 Założenia

Pojedyncze losowanie szumu w eksperymencie nr 1 nie wystarcza, żeby oszacować skuteczność przy wysokich poziomach. Dlatego dla poziomów 0–100% (krok 10%) wykonano po 50 niezależnych losowań szumu dla każdej z 26 liter, czyli 1300 rozpoznań na poziom. Te same funkcje co w programach (`render_letter`, `add_noise`, `Madaline`) wywoływano bezpośrednio w skrypcie `eksperymenty.py`.

3.2.2 Przebieg

Dla każdego rozpoznania zapisano: czy rozpoznanie jest poprawne, wyjście neuronu właściwej litery oraz największe wyjście spośród pozostałych 25 neuronów. Wyniki porównano z wartością teoretyczną (3), uśrednioną po literach. Obliczono też macierz podobieństwa wzorców WW^T (Rys. 4).

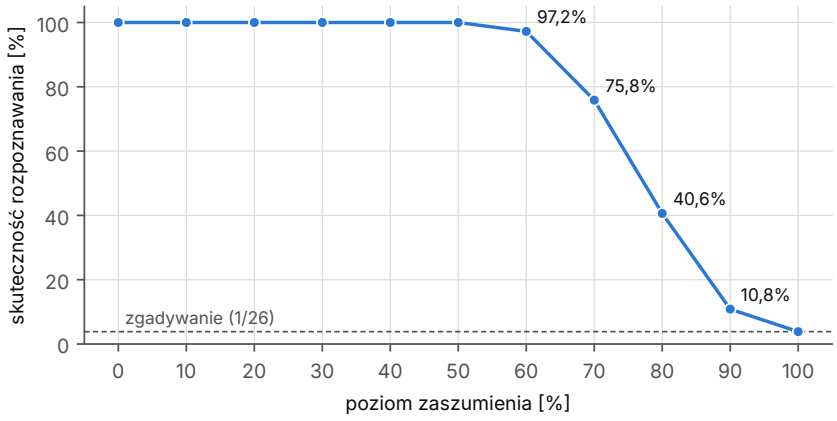
3.2.3 Rezultat

Tabela 2: Wyniki 50 powtórzeń dla każdego poziomu szumu (26 liter)

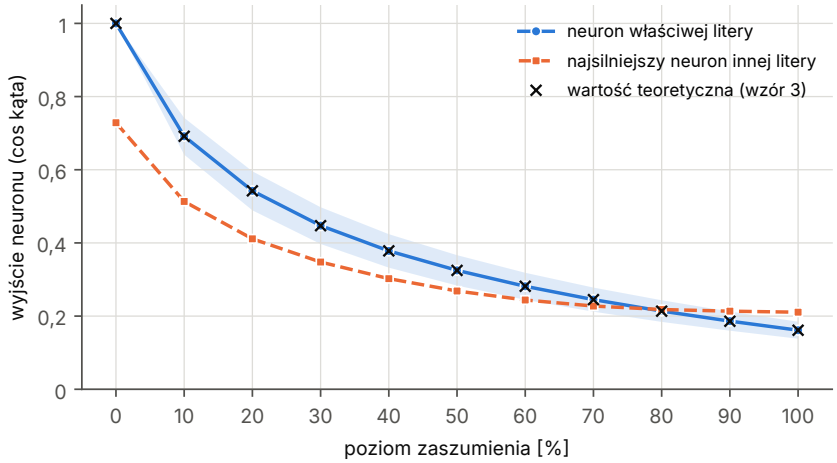
Szum	Skut. [%]	$\bar{y}_{\text{właściwy}}$	Wzór (3)	$\bar{y}_{\max \text{ inny}}$	Najczęstsze błędy
0%	100,0	$1{,}000 \pm 0{,}000$	1,000	0,729	–
10%	100,0	$0{,}692 \pm 0{,}050$	0,691	0,513	–
20%	100,0	$0{,}542 \pm 0{,}053$	0,542	0,411	–
30%	100,0	$0{,}447 \pm 0{,}050$	0,447	0,348	–
40%	100,0	$0{,}378 \pm 0{,}046$	0,378	0,302	–
50%	100,0	$0{,}325 \pm 0{,}042$	0,325	0,269	–
60%	97,2	$0{,}281 \pm 0{,}037$	0,281	0,244	r→m, i→m, r→h
70%	75,8	$0{,}245 \pm 0{,}033$	0,245	0,228	i→m, r→m, c→d
80%	40,6	$0{,}214 \pm 0{,}030$	0,214	0,218	i→m, n→m, z→m
90%	10,8	$0{,}186 \pm 0{,}026$	0,186	0,213	a→m, f→m, i→m
100%	3,8	$0{,}161 \pm 0{,}023$	0,161	0,211	a→m, b→m, c→m

Sieć rozpoznaje bezbłędnie wszystkie litery aż do 50% szumu (Tab. 2, Rys. 2). Pierwsze błędy pojawiają się przy 60% (skuteczność 97,2%), a potem skuteczność szybko spada: 75,8% przy 70%, 40,6% przy 80% i 10,8% przy 90%. Przy 100% wynosi 3,8%, czyli tyle, ile daje zgadywanie (1/26).

Wyjście neuronu właściwej litery jest zgodne ze wzorem (3) do trzeciego miejsca po przecinku (Rys. 3). Współczynnik pewności nie jest więc „pew-



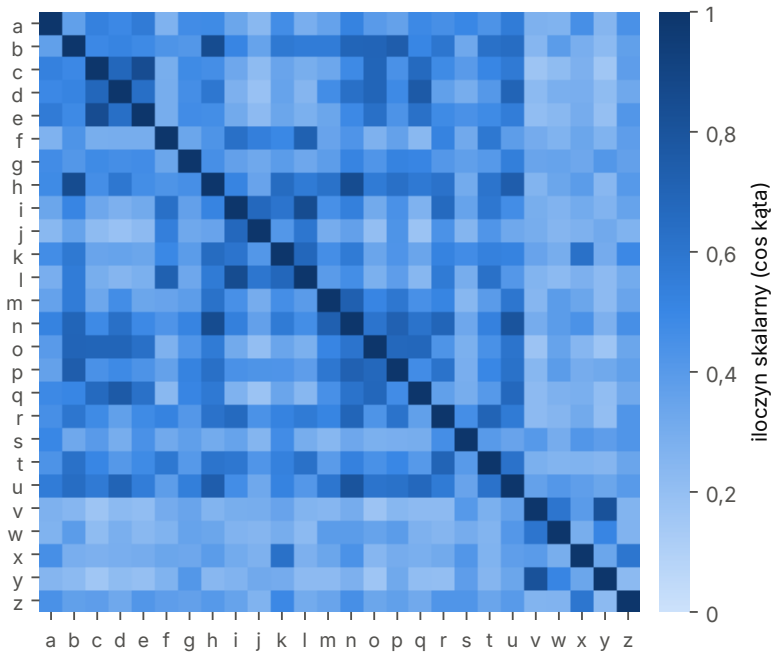
Rysunek 2: Skuteczność rozpoznawania w funkcji poziomu zaszumienia



Rysunek 3: Średnie wyjście neuronu właściwej litery (± odchylenie standardowe), najsilniejszego neuronu innej litery oraz wartość ze wzoru (3)

ścią" w sensie probabilistycznym, tylko przewidywalną funkcją poziomu szumu i liczby pikseli litery. Już przy 10% szumu spada do ok. 0,69, choć rozpoznanie jest wciąż w 100% poprawne. O poprawności decyduje **różnica** między wyjściem właściwego neuronu a najsilniejszego konkurenta. Bez szumu wynosi ona $1 - 0{,}729$, bo średnio najbardziej podobna inna litera ma cosinus 0,729. Wraz ze wzrostem szumu różnica maleje, a przy ok. 75–80% krzywe się przecinają. Odchylenie standardowe wyjścia ($\approx$ 0,03–0,05) sprawia, że błędy zaczynają się nieco wcześniej, od 60%.

Najbardziej podobne pary wzorców to `b`–`h` (0,851), `h`–`n` (0,849), `i`–`l` (0,849) i `c`–`e` (0,847) (Rys. 4). Mimo to błędy przy dużym szumie nie dotyczą głównie tych par. Litery przegrywają z `m`, które ma najwięcej czarnych



Rysunek 4: Podobieństwo (iloczyn skalarny) wzorców liter – macierz WW^T

pikseli (1462, wobec 535 dla `i`). Szum dodaje czarne piksele równomiernie na całym obrazie, a wkład tych pikseli do wyjścia neuronu j rośnie jak $q\sqrt{n_j}$. Szum faworyzuje więc wzorce „ciężkie". Pierwsze błędy to cienkie litery: `r`→`m`, `i`→`m`, a także `c`→`d`, bo `d` niemal zawiera kształt `c` i ma więcej pikseli. Przy 100% szumu we wszystkich 2000 próbach wygrało `m`, zgodnie z przewidywaniem $\sqrt{n_j/2N}$ ze Wstępu teoretycznego.

3.3 Eksperyment nr 3 – przesunięcie litery i zmiana kroju

3.3.1 Założenia

Sieć porównuje obrazy piksel po pikselu, więc sprawdzono, jak reaguje na zmiany inne niż szum. Znaki testowe były bez szumu, ale:

- przesunięte w poziomie o 1, 2, 4 lub 8 pikseli względem wzorca (parametr x),
- narysowane innym krojem o tym samym rozmiarze: Liberation Serif (krój o metryce zgodnej z Times), DejaVu Serif (inny krój szeryfowy) i Liberation Sans (bezszeryfowy).

3.3.2 Przebieg

Dla każdego wariantu rozpoznano 26 liter siecią zbudowaną z wzorców Times bez przesunięcia.

3.3.3 Rezultat

Tabela 3: Rozpoznawanie liter przesuniętych i w innym kroju (bez szumu)

Wariant	**Skut. [%]**	$\bar{y}_{\text{właściwy}}$	$\bar{y}_{\text{zwycięzca}}$	**Przykładowe błędy**
przesunięcie o 1 px	100,0	0,891	0,891	–
przesunięcie o 2 px	96,2	0,783	0,784	r→i
przesunięcie o 4 px	42,3	0,577	0,604	a→u, b→l, c→u, …
przesunięcie o 8 px	3,8	0,236	0,414	a→n, b→f, c→n, …
Liberation Serif	100,0	0,763	0,763	–
DejaVu Serif	23,1	0,433	0,511	a→o, b→f, c→t, …
Liberation Sans	38,5	0,562	0,624	a→d, c→u, d→o, …

Przesunięcie o 1 px nie powoduje błędów, ale obniża pewność do 0,891 (Tab. 3). Przy 4 px skuteczność spada do 42,3%, a przy 8 px do 3,8%, czyli poziomu zgadywania. Przesunięcie o 4 px bez żadnego szumu psuje więc rozpoznawanie bardziej niż 70% szumu losowego. Podobnie jest ze zmianą kroju. Liberation Serif, którego litery mają prawie te same kształty i szerokości co Times, rozpoznano w 100% (pewność 0,763). Krój bezszeryfowy (38,5%) i DejaVu Serif o innych proporcjach (23,1%) rozpoznawane są już słabo. Sieć MADALINE w tej postaci jest dopasowaniem do szablonu: rozpoznaje dobrze tylko znaki narysowane tym samym krojem i w tym samym miejscu co wzorce.

4 Wnioski

1. Zaimplementowana sieć MADALINE działa poprawnie: znaki identyczne ze wzorcami rozpoznawane są zawsze ze współczynnikiem pewności dokładnie 1,000.

2. Sieć jest bardzo odporna na losowy szum pikseli. Przy 10%, 30% i 50% szumu wszystkie 26 liter rozpoznano poprawnie (także w 50 powtórzeniach). Skuteczność spada dopiero powyżej 60%, a przy 90% (10,8%) zbliża się do poziomu zgadywania.

3. Współczynnik pewności jest kosinusem kąta między wektorami i maleje szybko nawet przy małym szumie (ok. 0,69 przy 10%). Jego wartość dobrze przewiduje wzór (3). Niska pewność nie oznacza złego rozpoznania. O wyniku decyduje przewaga właściwego neuronu nad najsilniejszym konkurentem.

4. Przy dużym szumie sieć myli się w sposób systematyczny: wybiera wzorce o największej liczbie czarnych pikseli (tu `m`), bo losowe czarne piksele zwiększają podobieństwo do dużych wzorców. Przy obrazie całkowicie losowym wynik zależy już tylko od liczby pikseli wzorców.

5. Słabością sieci jest brak odporności na przesunięcie i zmianę kroju. Przesunięcie o kilka pikseli albo inny krój czcionki szkodzą bardziej niż silny szum. W praktyce przed rozpoznawaniem należałoby wyrównać położenie i skalę znaku (np. wyciąć i przeskalować prostokąt otaczający literę) albo użyć wielu wzorców na jedną literę.

5 Załączniki

Pełny kod znajduje się w plikach `font_generator.py`, `madaline_ocr.py`, `eksperymenty.py` i `wykresy.py`. Poniżej podano najważniejsze fragmenty.
Szum (`font_generator.py`):

```
def add_noise(pixels, noise_level, rng):
    mask = rng.random(pixels.shape) < noise_level / 100
    noisy = pixels.copy()
    noisy[mask] = rng.integers(0, 2, size=int(mask.sum()))
    return noisy
```

Budowa sieci i rozpoznawanie (`madaline_ocr.py`):

```
def image_to_vector(path):
    pixels = np.asarray(Image.open(path).convert("L")) < 128
    return pixels.astype(np.float64).ravel(), pixels.shape
```

```
def normalize(v):
    length = np.linalg.norm(v)
    return v / length if length > 0 else v

class Madaline:
    def __init__(self, weights, labels):
        self.W = weights # wiersz l = znormalizowany wzorzec l
        self.labels = labels

    def outputs(self, x):
        return self.W @ x # y_l = w(l) . x dla wszystkich neuronow

    def recognize(self, x):
        y = self.outputs(x)
        winner = int(np.argmax(y))
        return winner, float(y[winner])

# w main():
for x, test_label in zip(X, test_labels):
    winner, confidence = net.recognize(x)
    print(f"{test_label} -> {net.labels[winner]}, confidence: {confidence
        :.3f}")
```

Bibliografia

[1] Materiały do Wykładu nr 1, Rozdział 4 – Sieć MADALINE do rozpoznawania znaków. Kurs Inteligentna Analiza Danych, Instytut Informatyki Politechniki Łódzkiej.

[2] Materiały do Wykładu nr 2, punkt 1.5 – Zadanie projektowe. Kurs Inteligentna Analiza Danych, Instytut Informatyki Politechniki Łódzkiej.

[3] Bernard Widrow and Michael A. Lehr. 30 years of adaptive neural networks: perceptron, Madaline, and backpropagation. *Proceedings of the IEEE*, 78(9):1415–1442, 1990.